

Document Number: P2891R0

Date: 2023-05-16

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2023-01-11 through 2023-05-10

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [January 11th, 2023](#)
- [January 25th, 2023](#)
- [February 1st, 2023](#)
- [February 22nd, 2023](#)
- [March 8th, 2023](#)
- [March 22nd, 2023](#)
- [April 12th, 2023](#)
- [April 26th, 2023](#)
- [May 10th, 2023](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)
- [P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08](#)
- [P2678: SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28](#)
- [P2766: SG16: Unicode meeting summaries 2022-10-12 through 2022-12-14](#)

January 11th, 2023

Draft agenda:

- Planning for Issaquah.
- [P2736R0: Referencing the Unicode Standard](#)
- [D2749R0: Down with "character"](#)

Attendees:

- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Mark Zeren
- Nathan Owen
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- A round of introductions was held to welcome Fraser as a new attendee.
- Planning for Issaquah:
 - Tom expressed interest in holding a meeting in Issaquah despite neither he nor Peter Brett planning to attend in person.
 - Tom reported that Steve agreed to facilitate the in-person aspects of the meeting in Issaquah.
 - Tom suggested aiming for a half day on Thursday.
 - Tom asked who planned to attend in person; three people expressed such intent.
 - MarkZ reported that he would have a conflict at 3pm every day.
 - Discussion ensued regarding the merits of reserving a room vs planning for in-person attendees to join via Zoom.
 - Steve noted that Zach's recent and upcoming papers may attract more interest.
 - Zach noted that SG16 tends to attract a few additional attendees beyond the regulars.
 - Tom stated he would request a room.
- [P2736R0: Referencing the Unicode Standard](#):
 - Tom summarized the discussion from the [2022-12-14 SG16 telecon](#) concerning the `__STDC_ISO_10646__` predefined macro.

- Corentin provided an introduction.
- Corentin reported having reviewed the terminology from the Unicode Standard to identify wording changes to be made.
- Corentin explained that ISO/IEC 10646 uses "character" in its wording, but that the proposed wording uses "abstract character" when appropriate.
- Corentin noted that "character" is retained for uses such as "character type".
- Hubert stated that the "abstract character" definition from the Unicode Standard can be broadly applied and thus should only be used when a broad interpretation is actually intended.
- Corentin replied that "abstract character" is relevant when mapping between different character sets.
- Corentin indicated that "abstract character" only ended up being used in one place.
- Jens asserted there is a need for a term to express equivalence between characters.
- Tom agreed and noted that "abstract character" is useful when there is more than one possible encoding for the same character; as in Unicode normalization forms.
- Hubert acknowledged that "abstract character" would be appropriate for the mapping of source code characters to the translation character set.
- Jens suggested that is a QoI concern since translation phase 1 behavior is implementation-defined.
- Hubert agreed use of the term could be avoided there if desired.
- Corentin explained that the C++ standard currently contains two normative references to ISO/IEC 10646, one of which is to an obsolescent version for a definition of UCS-2; the proposed wording replaces references to UCS-2 with a restricted form of UTF-16.
- Corentin stated that references to the Unicode Standard are inclusive of the annexes; normative references to the annexes are therefore removed.
- Corentin reported that many of the changes are mechanical; "UCS" was replaced with "Unicode".
- Corentin noted that the control code alias names table was removed following discussion on the SG16 mailing list.
- Steve pondered whether notes should be added to the C++ standard that explain where to find information in the Unicode Standard.
- Corentin replied that doing so can be useful; for example for `NameAliases.txt`.
- Hubert stated that "UCS Encoding Form" is more restrictive than "Unicode Encoding Form"; the latter isn't limited to the common UTF encodings..
- Mark asked if that terminology should not then be changed.
- Hubert replied that it might be necessary to add constraints or to find another way to identify the relevant encodings.
- Mark suggested that the first heading in Table 1 in [lex.charset] could be changed to "abstract character".
- Jens objected and explained that the first column lists code points and the second column lists names; these are concrete character references.

- Corentin replied that he had not felt a need to change the use of "character" in that table heading.
- Jens stated that the change to remove the control code alias table changes the specification with regard to allowances for the "BELL" and "ALERT" names.
- Corentin replied that wording was added to restrict alias names to those that are specified as "control", "correction", and "alternate".
- Jens noted that the code page chart appears to list "BELL" as a control name, but in a confusing way that is inconsistent with the names in `NameAliases.txt`.
- *[Editor's note: The discussion of "BELL" and "ALERT" centers around how the Unicode Standard presents the alias names for U+0007. The [C0 Controls and Basic Latin PDF](#) displays as follows.*

```
0007    <control>
        = BELL
```

[NameAliases.txt](#) is more clear in its intent:

```
# Note that no formal name alias for the ISO 6429 "BELL" is
# provided for U+0007, because of the existing name collision
# with U+1F514 BELL.

0007;ALERT;control
0007;BEL;abbreviation
```

]

- Tom summarized the concern; implementors might look at the code page charts and become confused or implement something other than what is intended.
- Corentin replied with doubts that implementors would base their implementations on the code page charts.
- Zach asked if the intent can be made more explicit by reintroducing a note to direct readers to `NameAliases.txt`.
- Jens replied that a note would be helpful and that he believes the existing table was originally built from content in `NameAliases.txt`.
- Steve suggested amending the proposed "control, correction, or alternate" wording to add "as specified in `NameAliases.txt`".
- Jens expressed concern about duplicating content from the Unicode Standard.
- Zach suggested retaining the prior "These names are derived from the Unicode Character Database's `NameAliases.txt`" wording but with "derived" removed.
- Corentin agreed to make a change.
- Jens stated that references to the Unicode Standard should have "the" capitalized.
- Tom expressed a preference in favor of "Unicode code point" over "Unicode scalar value" with the latter reserved for use in expressing requirements.

- Zach expressed indifference and noted that UCD properties won't be present for surrogate code points.
- Hubert stated that use of "Unicode scalar value" has the advantage of avoiding the question of whether non-scalar values need to be considered in the given context.
- Steve noted that "Unicode scalar value" implies a precondition that, if violated, could lead to undefined behavior.
- Jens suggested that references to chapters of the Unicode Standard should use both numbers and names for resiliency against changes.
- Corentin explained that references to UCS-2 were replaced with references to UTF-16 with additional restrictions to limit encoded characters to those in the BMP.
- Fraser replied that there is a semantic difference since UCS-2 allowed encoding surrogate code points.
- Corentin responded that the previous wording was not clear how such code points were to be handled.
- Hubert pointed out that, in the proposed wording for the `codecv_t` facets, the first bullet describes the artifacts produced by an encoding where as the second bullet names an encoding.
- Hubert stated that both bullets should be written such that they can be easily read as specifying encodings.
- Jens suggested retaining UCS-2 terminology by adding a definition of it that specifies it as a restricted form of UTF-16.
- Zach expressed a preference for the currently proposed wording with the category error corrected.
- Hubert suggested that the wording state that the facet only maps from that code point range and nothing else.
- Tom observed that, if the UTF-8 text has characters that map outside the BMP, the wording doesn't say what happens.
- Hubert stated that we need to make it clear that just converting to UTF-16 isn't acceptable.
- Corentin explained that he did not remove the UAX #31 reference since Steve is working on related changes.
- Corentin expressed uncertainty whether a separate UAX #31 reference is needed.
- Corentin observed that some unintended `tcode` LaTeX markup appears in one of the bibliography entries proposed for removal.
- Zach returned discussion to the `__STDC_ISO_10646__` predefined macro, noted that it is inherited from C, and opined that there is nothing to be done for it.
- Jens replied that the wording essentially states that `wchar_t` must be at least 21 bits for the macro to be defined.
- Hubert observed that the proposed change loses the requirement that storing a wide character stores a value that matches that character's Unicode scalar value.
- Hubert explained that this behavior is the design flaw that makes it not possible for a compiler to predefine this macro; the value held in an object of type `wchar_t` has a locale dependent interpretation.

- Zach suggested that restricting the implication to the encoding of wide character literals might be an improvement.
- Hubert suggested this might be a matter worth discussing in SG22.
- Zach asked if the wording could just defer to the C standard.
- Jens replied that we can do so for library wording, but not for core wording.
- Jens stated that we could match the wording in the C standard.
- Steve noted a misspelling of 10646 in the first paragraph of the Motivation section: "10446".
- [D2749R0: Down with "character"](#):
 - Discussion was postponed due to lack of time.
- Tom reported that the meeting will be on 2023-01-25 and will prioritize further review of P2736R0 and then D2749R0.

January 25th, 2023

Draft agenda:

- Planning for NOT Issaquah.
- [P2736R0: Referencing the Unicode Standard](#).
- [P2749R0: Down with "character"](#).

Attendees:

- Charlie Barto
- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Nathan Owen
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- A round of introductions was conducted for Nathan; though he had attended previous meetings, we had never formally introduced ourselves.
- Planning for NOT Issaquah.

- Tom explained that, due to competing priorities and a shortage of conference rooms, the only time slot available was for an evening session and, per previous discussion, an evening session time slot would be challenging for remote attendees; as a result, SG16 will not host an in-person meeting, but Tom is open to hosting another telecon next week before Issaquah to continue paper review.
- Zach agreed with meeting next week, but argued that an in-person meeting in Issaquah should still be held.
- PBrett opined that there would be few attendees.
- Corentin stated that a number of people that will have opinions on his papers will be present in Issaquah.
- Corentin asserted we should plan to meet in Varna.
- PBrett stated that Zach's [P2728R0: Unicode in the Library, Part 1: UTF Transcoding](#) and [P2729R0: Unicode in the Library, Part 2: Normalization](#) require SG16 review from an interface perspective.
- Tom agreed with Peter.
- Steve suggested that it would be useful to get early feedback from LEWG for Zach's papers.
- Steve noted that we don't want to spend months in review and then have LEWG question the design later.
- Corentin stated that we should ensure people are aware that these papers target C++26 and that time slots will be available in Varna and later meetings.
- Steve agreed that we should not host an official SG16 meeting in Issaquah.
- Tom stated that we will proceed with a telecon next week and no meeting in Issaquah.
- [P2736R0: Referencing the Unicode Standard:](#)
 - Corentin explained the changes made, as suggested by Jens, to provide a definition for UCS-2 as needed for `std::codecvt_utf8` and `std::codecvt_utf16`.
 - Corentin stated that the UCS-2 definition is specified in terms of scalar values so as to exclude lone surrogates.
 - Corentin noted that the previous wording did not address how lone surrogates were to be handled.
 - Hubert suggested moving where "only" appears in the UCS-2 wording.
 - Corentin reported that various grammar and spelling issues pointed out by Jens were corrected.
 - Corentin stated that the prior discussion of the `__STDC_ISO_10646__` predefined macro was inconclusive.
 - Hubert recalled a suggestion to treat this macro the same as `__STDC_VERSION__`.
 - Fraser asked if `__STDC_ISO_10646__` could be removed from the C++ standard and noted that implementations could still define it since it is a reserved identifier.
 - Corentin expressed reluctance to doing so as part of this paper since this paper is not intended to make design changes.
 - Corentin stated that he plans to work with SG22 and WG14 regarding the intended use of the macro.

- Corentin asserted that a minimal change suffices for now to remove the reliance on ISO/IEC 10646.
- Hubert replied that the proposed change isn't the minimal solution since it diverges from the C standard.
- Hubert clarified that the suggestion wasn't to reference the C standard, but rather to leave the definition rather meaningless so that implementors lean on C for meaning.
- Corentin asked if the suggestion was to just make the macro implementation-defined.
- Tom replied that he thought that was what Fraser had suggested.
- Fraser confirmed.
- Jens asserted that the wording should preserve the condition that the macro, if defined, has a value with a particular syntactical form.
- **Poll 1.1: Whether `__STDC_ISO_10646__` is predefined and if so, what its value is, are implementation-defined, retaining the mandated `yyyymmL` form.**
 - **Attendees: 11 (3 abstentions)**
 - **S F F N A S A**
6 2 0 0 0
 - **Unanimous consent.**
- Hubert noted some existing occurrences of "Unicode encoding" in the wording for `[lex.string.escaped]`.
- Corentin reported that he did not see a reason to update that wording.
- Hubert stated that the relevant encodings should be restricted to UTF-8, UTF-16, and UTF-32 because the formal definition of "Unicode encoding" is too inclusive.
- Discussion ensued regarding encoding form vs encoding scheme and the observation that, for `std::format` with a wide format string, `wchar_t` elements correspond to an encoding form.
- PBrett asked if the wording can just state "UTF-8, UTF-16, or UTF-32".
- Corentin agreed to make that change.
- Jens lamented the loss of wording in `[lex.name]` regarding what `XID_Start` and `XID_Continue` are and where to find their definitions.
- Jens asserted a note should be retained to direct readers to their definitions.
- Corentin opined that a note isn't needed since the terms are in a normative reference.
- Jens replied that such a note is present for other properties such as `Grapheme_Extend`.
- Corentin stated that he is fine with retaining a note.
- PBrett asked about following the existing pattern in `[format.string.escaped]` for the `General_Category` and `Grapheme_Extend` properties where it is stated "as described by table 12 of UAX #44".
- Zack expressed concern about the stability of text files.
- Tom suggested more generic wording like "as described in the Unicode character database".
- Jens agreed with that approach.
- Fraser noted that internet searches for `XID_Start` yield good results.

- Corentin edited the paper to update the wording.
- **Poll 1.2: Forward P2736R1, amended as discussed, to CWG and LWG as the recommended resolution of NB comments FR-010-133 and FR-021-013.**
 - **Attendees: 10 (1 abstention)**
 - **SF F N A SA**
 - **7 2 0 0 0**
 - **Unanimous consent.**
- [P2749R0: Down with "character"](#):
 - *[Editor's note: D2749R0 was the active paper under discussion at the telecon. The agenda and links used here reference P2749R0 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Corentin provided an introduction:
 - The wording substitutes "Unicode scalar value" for "character" in many places, but retains the latter in some contexts.
 - This removes the "translation character set" indirection.
 - The changes were mechanically applied.
 - PBrett expressed support for the improved specificity.
 - Corentin began reviewing the wording changes.
 - Tom noted that Jens had previously requested an overview of the final state we are driving towards with these kinds of changes.
 - Corentin replied that the motivation section addresses some of those concerns.
 - Jens stated that he finds the proposed wording confusing since "character" ends up getting mixed in with Unicode terminology.
 - Corentin explained that he has concerns about introducing a lot of churn that doesn't help to improve clarity.
 - Fraser stated that additional specificity is probably needed to clarify which characters constitute new-line and whitespace.
 - Jens noted that, with the exception of new-line, that characters are specified using Unicode code points.
 - Jens noted that [P2348 \(Whitespaces Wording Revamp\)](#) is relevant.
 - Corentin explained that he intentionally did not modify the specification of whitespace in this paper.
 - Corentin expressed a desire for agreement on the replacement of "elements of the translation character set".
 - PBrett noticed an editorial issue in [lex.charset] for *n-char*; the updated wording retains "set" from the intended substitution of "Unicode code point" for "member of the translation character set".
 - Jens observed that a change in [lex.phases] to substitute "Unicode scalar value" for ".. elements of the translation character set" retains a reference to [lex.charset] that no longer makes sense.
 - Jens opined that the note in [lex.charset] that describes the difference between "code points" and "scalar values" should be retained.

- Hubert stated that the location of that note is a little odd since it doesn't encapsulate the notion of "abstract character".
- Hubert observed that the updates to [lex.phases]p3 only updated one of the two uses of "space character".
- Corentin asked for opinions regarding a footnote in [lex.name] that discusses representation of characters outside the basic character set in external identifiers.
- Hubert stated that the footnote could use some updates.
- Tom opined that the footnote should just be removed.
- Jens noted that [\[implimits\]p\(2.6\)](#) does specify a minimum limit for the number of significant characters in an external identifier.
- PBrett stated that the the uses of "character" in that annex need to be addressed.
- Corentin asked if CWG would be content with the removal of that footnote.
- Jens replied that he does not know but that he personally does not see value in retaining it; the note probably serviced its value 20 some years ago.
- Jens observed that "characters" would have to be interpreted as code points for the purposes of the external identifier limit.
- Tom noted the implication that, if UTF-8 was used for the encoding of external identifiers, a worst case limit must be assumed such that a limit of 1024 code points implies a limit of 4096 code units.
- Tom reported that SG16 will meet in one week, on 2023-02-01 in order to squeeze in one more review of P2749R0 before the WG21 meeting in Issaquah.

February 1st, 2023

Draft agenda:

- [P2749R0: Down with "character"](#).

Attendees:

- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- Mark de Wever
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- Tom announced that this telecon is SG16's 100th!
- Corentin noted that LEWG is considering hosting an evening session in Issaquah to discuss Zach's [P2728R0: Unicode in the Library, Part 1: UTF Transcoding](#) and [P2729R0: Unicode in the Library, Part 2: Normalization](#) papers.
- Steve expressed support for early LEWG review so as to avoid a situation in which SG16 forwards a paper with interfaces that LEWG does not approve of; such cases have occurred with other study groups.
- PBrett noted that there are sometimes competing perspectives between what domain experts value and what LEWG values.
- PBrett acknowledged the possibility that LEWG will approve of Zach's design, that SG16 proceeds with making changes during its review, and that LEWG finds that it does not approve of the changed direction.
- [P2749R0: Down with "character"](#):
 - *[Editor's note: D2749R0 was the active paper under discussion at the telecon. The agenda and links used here reference P2749R0 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Tom provided a summary of the last telecon.
 - Tom raised two concerns to be addressed.
 - Comments that Jens raised in a [post to the SG16 mailing list](#).
 - Whether the paper should take a dependency on [P2348 \(Whitespaces Wording Revamp\)](#).
 - Corentin explained that, with regard to Jens' concern about inconsistent use of the "Unicode code point" and "character" terms, that the changes made to mechanically replace "character" in 10-20 pages of wording were quite extensive.
 - Corentin stated that, in cases where the wording refers to a specific character, such as U+0020 SPACE, that the term "character" is appropriate.
 - Corentin acknowledged Jens' concerns, but noted that the updated wording does reduce ambiguity.
 - Corentin claimed that the proposal includes a minimal change and that additional changes could be done editorially at a later time.
 - Tom reported having spent time reviewing the changes and that he found the various uses of "Unicode scalar value", "Unicode code point", and "character" rather confusing.
 - Tom expressed concern that the differences are subtle and that it might be unfair to place the project editor in the position of having to deal with those differences; at least not without clearly specified guidelines.
 - PBrett responded that the project editor shouldn't make such changes since they can have normative impact.
 - Corentin reiterated that his goal with the paper is to remove the "translation character set" terminology in C++23 to avoid its appearance in new teaching materials.

- Tom suggested modifying the paper title to append "in lexing" since that better matches the scope of the proposed changes.
- Tom suggested reviewing the wording to ensure consistent terminology use.
- The group started reviewing the changes to [lex.phases].
- PBrett noted that people complain about the Unicode terms, but that their use is well justified in an international standard.
- PBrett noted the use of "character" in association with new-line and asked whether new-line could consist of multiple code points.
- Tom responded that, since the translation input is now specified in terms of Unicode code points, that we can define exactly what a new-line character is.
- Hubert agreed and stated that would provide a better basis for defining whitespace.
- Steve noted that `\n` can have platform impact in some contexts but that it doesn't in lexing.
- Corentin replied that more significant changes would be required to substitute U+000A LINE FEED for "new-line character" and that there would still be remaining uses of "character".
- PBrett stated it could be a conscious choice to leave those cases to a later paper like P2348.
- Hubert responded that the motivation for asking for additional work is to avoid increasing internal friction within the standard wording as the paper does now.
- Corentin expressed concern regarding incorporating work from P2348 due to concerns CWG had with that paper.
- Steve noted that, in lexing context, specifying new-line is not observable.
- Fraser asked if new-line characters are observable in raw string literals.
- Hubert replied negatively and explained that, in a raw string literal, new-line is mapped to `\n` in the execution character set.
- Steve stated issues with that were fixed.
- Tom stated that there are related CWG issues.
- [*Editor's note: See [CWG issue 1655 \(Line endings in raw string literals\)](#) and [CWG issue 1709 \(Stringizing raw string literals containing newline\)](#).]*
- Corentin pondered whether it is necessary to consider both papers at the same time.
- Hubert recommended sending a message to the CWG mailing list to ask about that.
- Corentin noted the substitution of a Unicode code point reference in place of "backslash character".
- Corentin mentioned that the changes are intended to allow "code point" to be used for non-Unicode character sets; hence "Unicode code point" is used in contexts specific to Unicode.
- Tom expressed uncertainty about the addition of "abstract" in [lex.phases]p1.
- Corentin responded that abstract characters are used to explain mapping between character sets.
- Tom agreed, but stated that wording is missing to tie the input to a sequence of abstract characters.

- Fraser suggested adding a statement to specify that the lexer processes a sequence of Unicode code points.
- Hubert noted that the current wording limits the effort required to document how input is mapped to a sequence of characters.
- Corentin acknowledged that wording could be added to state that the implementation-defined mapping produces a sequence of Unicode scalar values.
- Corentin pointed out how "space character" and "whitespace character" is used differently in [lex.phases]p3.
- Mark observed that the changes to [lex.phases]p3 substituted "multi-Unicode code point" for "multi-character".
- Hubert suggested substituting "token comprising multiple Unicode code points" instead.
- Tom expressed support for substituting "U+0020 SPACE character" for "space character".
- Corentin observed that "character" can be omitted in the substitution.
- Hubert agreed.
- Tom suggested that, in [lex.charset], "code points" could be substituted for "characters" in "The basic character set is a subset ... consisting of 96 characters".
- Hubert replied that such a substitution would introduce a category error; the basic character set is intended to be a set of abstract characters and is used as such in some places.
- Tom suggested "The basic character set is a subset of the abstract characters included in the Unicode character set".
- Hubert suggested a simpler change; "the basic character set consists of the 96 characters specified in [lex.charset.basic]".
- Tom stated that the changes to the note in [lex.charset]p6 should state "Unicode code point" instead of just "code point" for consistency.
- PBrett opined that, with only 15 minutes remaining, that he did not think we'll be ready to forward the paper.
- Tom agreed and stated that he doesn't feel that we have sufficiently reviewed the paper to be confident in polling it.
- Tom noted that our goal is not to make a perfect standard, but rather to make improvements; we can poll forwarding it knowing that more work will be needed.
- PBrett suggested polling to continue work and then poll a recommended resolution for the related NB comment in C++23.
- Steve asked what the motivation is for getting this change in C++23.
- Corentin responded that he is motivated to remove "translation character set" before it appears in training materials.
- **Poll 1.1: P2749R0 "Down with 'character'" should be included in the IS only if the updates to whitespace specification described in P2348 "Whitespaces Wording Revamp" are also included.**
 - Attendees: 7 (1 abstention)
 - SF F N A SA

0 4 1 0 1

- Weak consensus.
- Poll 1.2: Forward P2749R0 "Down with 'character'", revised as discussed, to CWG for C++23 as the recommended resolution of ballot comment FR-020-014.
 - Attendees: 7
 - SF F N A SA
1 1 3 2 0
 - No consensus.
- Poll 1.3: Recommend rejection of ballot comment FR-020-014 as no consensus for change.
 - Attendees: 7 (1 abstention)
 - F N A
4 1 1
 - Consensus.
- Discussion ensued regarding how to prioritize papers for review following the Issaquah meeting and ended with the following tentative prioritization schedule:
 - [P2741R1 \(user-generated static_assert messages\)](#).
 - [P2758R0 \(Emitting messages at compile time\)](#).
 - [P2773R0 \(Considerations for Unicode algorithms\)](#).
 - [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#).
 - [P2729R0 \(Unicode in the Library, Part 2: Normalization\)](#).
 - [P2348R3 \(Whitespaces Wording Revamp\)](#).
 - [P2749R0 \(Down with "character"\)](#).
- Tom announced that the next meeting will take place on 2023-02-22.

February 22nd, 2023

Draft agenda:

- [P2773R0: Considerations for Unicode algorithms](#).

Attendees:

- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Nathan Owens
- Peter Brett
- Robin Leroy

- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2773R0: Considerations for Unicode algorithms](#):
 - PBrett suggested that Corentin provide a brief introductory overview by speaking to each of the points in the "TL;DR" section.
 - Corentin presented an overview:
 - The paper is guided by experience obtained through prototyping.
 - The paper is informed by expectations of performance requirements.
 - Views are a good fit for Unicode algorithms.
 - It is not clear what invariants would be desirable for a new text container.
 - Unicode does not impose constraints on what code points may follow other ones; new code points can be inserted effectively anywhere.
 - The paper assumes the availability of transcoding interfaces.
 - Normalization should be an early focus.
 - The existing interfaces for uppercase and lowercase operations in the C++ standard do not suffice to address the needs of all languages.
 - The existing interfaces for case insensitive operations in the C++ standard are likewise insufficient.
 - Text operations need to be easy for programmers to use with UTF-8 encoded text.
 - Interfaces for text operations should not be replicated for every encoding of Unicode code points.
 - It should be possible to, for example, perform casing and normalization operations without having to materialize a code point sequence in between the operations.
 - The standard lacks localization facilities sufficient to support tailoring.
 - Interfaces that support tailoring should be implemented using ICU4X.
 - Interfaces for the non-tailored algorithms may be provided as `constexpr` since they are locale invariant.
 - Providing views implemented with ICU would be challenging and would limit performance opportunities and iterator categories.
 - Interfaces that support tailored algorithms with an interface similar to that used for non-tailored interfaces can be added later.
 - Non-tailored interfaces are useful despite their being insufficient for user presentation purposes in general.
 - There are no known use cases for performing normalization for non-Unicode encodings like EBCDIC or Shift-JIS.

- Code unit sequences should be validated by default on consumption; for safety reasons.
 - Standardized interfaces should not be constrained by what ICU is capable of providing.
 - Implementation via ICU should be supported when doing so doesn't limit potentially better solutions.
 - Implementation via ICU prohibits `constexpr` and `noexcept`.
 - Standardized interfaces should take advantage of ranges and views.
 - Size hints are useful for reserving memory in order to avoid reallocations.
 - It is important and reasonable to optimize table lookups for most character properties.
 - In place mutation of text does not perform well.
 - UTF decoding and encoding is inexpensive even when not optimized; it might not be necessary to design every interface to support maximum optimization possibilities.
- Zach stated that normalization is a relatively expensive operation and is therefore best suited to eager transformations.
 - Victor commented in the chat that lazy operations often limit optimizations like SIMD.
 - Jens also commented in the chat that range filters are probably at odds with SIMD, noted that table lookups aren't SIMD-friendly, and pondered how much the Unicode algorithms benefit from SIMD.
 - Zach noted that maintaining multiple levels of enclosed iterators can be painful.
 - Zach reflected on prior discussions regarding requirements to be able to implement Unicode functionality using ICU and asked how important such implementability is.
 - *[Editor's note: [P1238R1 \(SG16: Unicode Direction\)](#) lists as a constraint that implementors cannot afford to rewrite ICU.]*
 - Tom responded that his prior statements were partially motivated by politics; a desire to assure implementors that SG16's efforts would not culminate in a requirement for them to implement all Unicode functionality on their own.
 - PBrett indicated that he is not worried regarding implementability via ICU.
 - Tom noted that the pipeline approach presented in the paper allows combining transformations in ways that may be order dependent.
 - Tom asked Robin to confirm that case folding and normalization operations are order dependent.
 - Robin noted that case folding does not depend on tailoring and confirmed that there are order dependencies; `toCasefold(toNFKC(S))` does not produce the same result as `toNFKC_Casefold(S)`.
 - *[Editor's note: In later correspondence, Robin noted that Turkic case folding (`İ -> i, i -> İ`) is usually not performed for non-Turkic languages as noted in [CaseFolding.txt](#).]*
 - Robin stated that, for canonical normalization, it is common to normalize at program boundaries, but that compatibility normalization might need to be done locally.
 - Tom noted the implication; programmers can't expect to combine transformations arbitrarily and get the "right" result.

- Jens replied that this means special algorithms are needed for some transformations.
- Jens stated that the range pipeline approach is idiomatic and seems amenable to these transformations.
- Jens noted that ranges was transformational in its ability to avoid the need for intermediate storage.
- Jens cautioned that this ability comes with the cost that the transformations be interruptible.
- Jens expressed appreciation for the syntax that views enable but that he would like to understand the performance trade off with respect to eager transformations.
- Jens asserted it would be useful to have some performance data in a paper.
- Jens acknowledged that normalization as a view adapter would require some local memory, but noted that might be cache advantageous.
- Steve expressed concern over including normalization as a pipeline stage; if normalization iterators have to bounce between various buffers, performance may suffer.
- Jens noted that views can be materialized when it is advantageous to do so.
- PBrett reported that he has applications for normalization views.
- Corentin stated that it is probably advantageous to materialize a view if it will be iterated more than once.
- Corentin reported having implemented a normalization view and that he did not find the implementation to be too difficult.
- Corentin noted that normalization can benefit from limits like those specified in the stream-safe text format.
- *[Editor's note: The stream-safe text format is described in [UAX #15 \(Unicode Normalization Forms\)](#).]*
- Corentin commented that normalization can be expensive, but only when transformations are actually necessary; characters corresponding to some of the `General_Category` classes can be recognized and copied without further lookup.
- Victor stated that, with regard to implementability with ICU and the benefits of range based interfaces, good performance is more important.
- Victor advised creating a reference implementation so that performance can be evaluated relative to ICU.
- Victor noted that we do not want another experience like `std::regex` where the implementations available exhibit poor performance.
- Zach reported having compared performance between his [Boost.Text](#) implementation and ICU and found his implementation initially lagged ICU performance by 50 times.
- Zach explained that he was able to improve performance after studying the implementation in ICU, but that his implementation was still 10 times as expensive as ICU.
- Zach asserted that we should not expect implementors to match ICU performance.

- Zach declared that we don't want to specify a facility that will pose a decade long implementation challenge as happened with `std::from_chars()` and `std::to_chars()`.
- Zach noted that the lazy range approach can be implemented using ICU.
- Zach suggested that we can specify both eager and view based implementations.
- PBrett provided a use case for lazy normalization; Unicode regular expression matching can perform better with NFKD normalized text and it can be advantageous to only normalize until a match is found.
- Corentin replied to Victor's request for a reference implementation that he would make his prototype work available, but that better implementations are possible.
- Corentin expressed skepticism that there isn't room to improve on ICU performance.
- Tom asked Robin if the CLDR follows [BCP-47](#).
- Robin replied that they are maintained in sync and that the CLDR uses some extensions defined in BCP-47.
- Robin stated that, with regard to standardizing localization support, localization is fundamentally not stable since it tracks evolving human behavior.
- Robin noted that, since the C++ standard is updated every 3 years, it will always trail localization changes.
- *[Editor's note: Robin noted in later offline discussion that CLDR releases occur at least every six months.]*
- Tom asked if some of the locale properties specified in BCP-47 are stable.
- Robin replied that BCP-47 specifies a syntax and that it has some stability assurances.
- *[Editor's note: Robin provided additional detail in offline correspondence following the telecon; The relation between Unicode language and locale identifiers and BCP-47 is documented in the "BCP 47 Conformance" section of [UTS #35 \(Unicode Locale Data Markup Language \(LDML\)\)](#) and stability is discussed in the "Stability of IANA Registry Entries" section of BCP-47 in [RFC 5646](#). The [ISO 3166](#) standard responsible for assigning country codes also implements a transition policy for changes to country codes as described at https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2#Transitional_reservations.]*
- Corentin recalled discussion of Zach's papers in Issaquah and noted that the level of stability guarantees is not sufficient for ABI purposes.
- Robin reported that the line breaking algorithm changes frequently but that other algorithms, like grapheme breaking, change less frequently.
- Robin advised focusing on use cases when considering the Unicode algorithms as different use cases may have different concerns.
- Robin noted that case folding has stability concerns and that it is stable for NFKC normalized text.
- Zach pondered whether it is reasonable to standardize something like a case-insensitive search.
- Tom reported that the next meeting will be 2023-03-08 and that we will continue discussion of this paper.

March 8th, 2023

Draft agenda:

- [P2773R0: Considerations for Unicode algorithms.](#)

Attendees:

- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Nathan Owen
- Peter Brett
- Robin Leroy
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [P2773R0: Considerations for Unicode algorithms:](#)
 - PBrett introduced the agenda.
 - Robin provided an overview of tailoring:
 - *[Editor's note: Robin shared his notes following the meeting; they can be found [here](#).]*
 - Points 3 and 5 of the "TL;DR" portion of the paper are related to tailoring.
 - Normalization is noted as a priority in the paper and that is a good thing as there are examples of languages and products that do not handle normalization well.
 - The Unicode standard provides examples of tailoring, but those examples are not prescriptive.
 - Tailoring does not mean "language dependent"; it permits many kinds of transformations.
 - Case folding has exactly one case of language specific behavior; Turkic case folding.
 - "I" -> "i" (U+0049 LATIN CAPITAL LETTER I -> U+0131 LATIN SMALL LETTER DOTLESS I)
 - "İ" -> "i" (U+0130 LATIN CAPITAL LETTER I WITH DOT ABOVE -> U+0069 LATIN SMALL LETTER I)
 - ICU supports this behavior via a boolean flag on case folding interfaces.

- Case folding was intended to support identifier equivalence.
 - NFKC case folding does not include language specific tailoring.
 - UAX #29 provides examples of language-dependent grapheme cluster tailoring, but at present, that isn't done in practice.
 - The CLDR technical committee is experimenting with a language-independent tailoring of grapheme clusters with a different behavior for Indic scripts. That work has not been forwarded to the UTC yet, but is intended to eventually be used as the new default default behavior.
 - Changes to [UAX #29 \(Unicode Text Segmentation\)](#) are likely to be proposed.
- ICU supports tailoring for line breaking beyond what is stated in the Unicode Standard:
 - Support for dictionary based layout for Thai.
 - Machine learning based layout for Burmese, Chinese, Japanese, and Thai.
 - Different line breaking rules for numbers.
- Line breaking behavior can be script based as opposed to language based.
- Case mapping includes language specific tailoring.
- Collation is used for sorting, but is also used for case insensitive search:
 - The French word "ŒUF" is primary-equal to "oeuf" for collation, but case-folding would not produce a match.
 - The German word "fuer" matches "für" in German, but not in French.
- Fraser shared in chat: "my favourite tailoring (for collation) is that in traditional name collation in Scotland, surnames starting "Mc" (e.g. McAdam) sort as if they begin "Mac" (so McAdam and MacAdam sort equivalently)."
- PBrett noted that, in some Scandinavian dictionaries, words beginning with Å (U+00C5 LATIN CAPITAL LETTER A WITH RING ABOVE) get ordered differently when they correspond to a place name, but not otherwise.
- PBrett added that, in Japanese, kanji have different meaning if they form part of a person's name as opposed to part of another word.
- Corentin asked for examples of production systems that handle these cases correctly.
- Robin replied that collation tailoring doesn't generally perform natural language processing, but noted that locale specifiers may include extensions for phone book order, dictionary order, or other ordering forms.
- PBrett stated that it is common for programmers to want to sort in many different ways and shared examples of ordering by surname vs address.
- Robin replied that the CLDR provides data for some of those applications and provided an example of ordering contacts for display on a phone.
- Robin noted that the non-tailored collation algorithm will produce incorrect results for many languages but will still be more accurate than sorting by, for example, code point values.

- Corentin asserted that collation support is necessary, but that it should not be considered high priority for now.
- Corentin noted that support for collation is responsible for a significant proportion of the data included with ICU.
- Robin stated that Swift supports non-tailored EGCs and both default and language dependent collation and case mapping.
- Robin noted that Swift encountered some challenges with their EGC segmentation and regional indicators but have not, in general, had issues with instability and [UAX #29 \(Unicode Text Segmentation\)](#).
- Robin reported that, with highly unstable data, programmers will generally want to opt-in to updates independently of compiler upgrades.
- Tom observed that might be an argument for providing Unicode versioned interfaces in some cases.
- Robin agreed, but advised doing so only for highly unstable data.
- Corentin asked about governance of the CLDR:
 - Is there a specification?
 - Is there a stability policy?
 - Could the CLDR be referenced from the C++ standard?
- Robin replied that he does not think the CLDR has a stability policy.
- Robin stated that the CLDR is released every six months, that it moves fast, and that it is maintained by its own committee within the Unicode Consortium.
- Robin noted that the syntax used for the CLDR data is standardized via [UTS #35 \(Unicode Locale Data Markup Language \(LDML\)\)](#).
- Tom asked if UTS #35 could be used as a specification to implement an interface to the CLDR as a data source.
- Robin replied that it could be in principle but that compatibility problems might arise if the CLDR version is not aligned with the implemented version of UTS #35.
- Corentin asked Robin if he agreed that it would be useful to add support for non-tailored case transformations in the near term.
- Robin replied that doing so would be a considerable improvement over the status quo.
- Robin noted that programmers that use the locale independent interfaces where they should not will attract appropriate attention from the internationalization proponents within their organization.
- PBrett observed that there is a segment of programmers that might not care about collation being correct for various locales.
- Corentin agreed and noted that ICU must be used today for locale specific support and then observed that having locale independent interfaces might prompt programmers to do the right thing.
- Robin asserted that providing non-tailored interfaces would be beneficial and that the standard should note their appropriate use.
- PBrett asked for comments regarding how tailored interfaces should be provided.
- Corentin replied that tailoring has different requirements and requires different types.

- Corentin suggested that such types can perhaps be hidden from programmers; for example, views that adjust the types used based on provision of a locale object.
- PBrett asked if tailoring should be expressed via locale facets or if it is sufficiently disjoint from locale that it should have its own facility.
- Corentin opined that we should not further build on `std::locale`, but that an interface that accepts some kind of locale object to opt-in to tailoring without other customization is possible.
- Corentin noted that ICU4X allows customization via "providers" in addition to locale-based tailoring.
- Corentin expressed uncertainty whether customization beyond locale-based tailoring should be provided.
- Robin replied that tailoring is unbounded and thus too vast a concept to be used in an unqualified manner.
- Robin stated that support for [UTS #35 \(Unicode Locale Data Markup Language \(LDML\)\)](#) would likely be required to do more, but that probably isn't feasible for the near future.
- Corentin stated that `std::locale` facets support customization but that the design can't possibly account for all tailoring possibilities.
- Corentin expressed a preference for an implementation-defined locale-based design.
- Jens stated that tailoring appears to mean a lot of things, that it isn't clear why or how we would provide an interface, and that we are unlikely to specify use of an AI for line breaking.
- Jens opined that tailoring appears to go beyond the existing facilities that do things like replacing "." with "," when formatting numbers.
- Jens asserted that programmers can insert their own transformations and that the standard does not have to provide support other than via interoperability.
- Jens opined that there is not a need to invest time in that direction now.
- Jens agreed that there are likely cases that can be plausibly and meaningfully provided based on locale.
- Jens expressed skepticism that we'll pursue a replacement for `std::locale` in the near term.
- Jens expressed support for specifying some tailored algorithms when the tailoring can be specified with few parameters.
- Hubert shared a perspective that `std::locale` facets are extensions of what C provides.
- Hubert stated that it might have been a mistake to make `std::locale` extensible, but such extensions remain an option subject to the limitation that these objects are tied to the C locale ID.
- Tom asked if there is reason to believe that it is not sufficient for programmers to be able to insert their own tailored transformations in pipelines that they construct.
- PBrett pondered whether the standard library should include non-DUCET data.
- *[Editor's note: The Default Unicode Collation Element Table (DUCET) is described in [UTS #10 \(Unicode Collation Algorithm\)](#).]*

- Jens asked why we would provide data if we don't provide algorithms that use it.
- PBrett replied that the availability of the data could enable programmers to use it to provide their own tailoring.
- Corentin responded that, given an expectation that programmers provide their own algorithms, there is nothing to provide.
- Corentin noted that the more we provide, the greater the risk that we'll have to break it later.
- Corentin asserted that the CLDR data cannot be consumed in a similar manner to timezone data.
- Tom suggested that such consumption should be possible with an implementation of the LDML.
- Corentin responded that providing such an implementation is equivalent to implementing ICU.
- Corentin shared an understanding that ICU's complexity is primarily due to integration with the CLDR.
- Corentin expressed our task as determining how implementors can provide CLDR-based tailoring using ICU or ICU4X.
- Corentin noted the implication with regard to portability and suggested ICU4X might provide a better building block.
- Robin noted his own association with ICU4X and expressed its goal to improve portability based on a more flexible and modular design.
- Robin stated that ICU4X does not offer strong stability guarantees.
- Robin agreed with the perspective that implementing support for LDML is tantamount to implementing half of ICU.
- Hubert noted that the existing C++ collation support is not customizable and that it simply exposes what is provided by the C library.
- Jens suggested it might be feasible to provide the DUCET data in a pre-compiled form.
- *[Editor's note: doing so would potentially side step the LDML implementation concerns.]*
- Jens noted that, for collation, we all have an intuitive understanding that languages work differently, but that such understanding is less clear for other algorithms.
- **Poll 1: Papers proposing standard library Unicode algorithms should include a section discussing future extensions to support Unicode locale-based tailoring.**
 - **Attendees: 10 (2 abstentions)**
 - **SF F N A SA**
3 5 0 0 0
 - **Unanimous consensus in favor.**
- **Poll 2: It is a design goal that non-tailored standard library Unicode algorithms be efficiently implementable using ICU.**
 - **Attendees: 10 (2 abstentions)**
 - **SF F N A SA**

1 2 1 3 1

▪ **No consensus.**

- Tom pondered concerns regarding ABI, ODR, and support for constant evaluation.
- Corentin noted that we haven't discussed whether we want to support constant evaluation or not.
- Corentin stated that the relevant ODR concerns are similar to those for `std::source_location` and that we strongly don't care about such violations.
- Hubert expressed a desire to, from an implementation perspective, place much of the associated data in shared libraries and not have them exposed via header files.
- Corentin suggested discussion is needed regarding freestanding support and how that impacts support for header-only implementations.
- Tom announced that the next meeting will take place on 2023-03-22 and that we'll start reviewing Zach's [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#).
 - PBrett noted that Zach's paper will provide additional fodder for discussing use of `char32_t` as a Unicode code point type and exposure of Unicode algorithms as views.

March 22nd, 2023

Draft agenda:

- [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#).

Attendees:

- Charles Barto
- Corentin Jabot
- Fraser Gordon
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark de Wever
- Nathan Owens
- Peter Bindels
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#):
 - Zach provided an introduction:
 - The proposal provides interfaces to facilitate conversion between the UTF encodings.
 - The intent is to provide support for ranges and iterators equally well.
 - The proposed interfaces allow for use of SIMD operations with contiguous iterators.
 - Transcoding algorithms are also proposed.
 - Special accommodations are provided for easy use of null-terminated strings.
 - The ubiquity of string literals and pointers to `char` motivates specialized interfaces.
 - The proposed interfaces constrain code unit types based on size, not particular character type.
 - Code unit types could also be constrained to particular character types but algorithms are generally not so constrained; this design is more general.
 - Most programmers are expected to use higher level interfaces, but these lower level interfaces enable specialized behavior; e.g., for text chunking.
 - A `null_sentinel_t` type and a corresponding `null_sentinel` variable enable generalized support for null terminated strings.
 - Additional utilities like `is_encoded` and `find_invalid_encoding` provide nice-to-have functionality.
 - The `transcode_to_utf8` and similar functions could be expanded to perform UTF validation without transcoding.
 - Transcoding iterators are provided for converting between UTF encodings.
 - Error handling is configurable, but only one error handling policy is currently specified; `use_replacement_character`.
 - Insert, front-insert, back-insert, and output transcoding iterators are specified with associated factory functions.
 - It is expected that the transcoding views would be most frequently used.
 - The `as_utf` family of functions return a type that, given a sequence of code units in one UTF encoding, provides a view of code units for another UTF encoding.
 - Formatter specializations are provided for the views.
 - The transcoding iterators wrap a range, not just a single iterator, because advancing the iterator might advance the underlying iterator multiple times thus requiring a sentinel to recognize the end of the underlying range.
 - Wrapping a transcoding iterator in another transcoding iterator yields a transcoding iterator from the base UTF encoding to the final one; this avoids accumulating multiple levels of wrapped iterators without requiring type erasure.
 - Fraser reported in chat that there is a stray single quotation mark in the `requires` expression in one of the `operator==` declarations for `utf_8_to_16_iterator` in section 4.6.1, "First, the basic ones".

- Jens stated that the proposed factory functions are probably not needed if support for class template argument deduction (CTAD) can be provided.
- Zach replied that he added them to follow existing precedent in the standard but agreed they could be omitted.
- Tom expressed surprise regarding the proposed behavior of `as_utf8()`; he expected a function with that name to provide a view that decodes UTF-8 from underlying data in order to produce code points.
- Zach replied that the proposed `as_utf8()` design provides a view of UTF-8 code units (not code points) by transcoding from whatever encoding the underlying data is encoded in.
- Jens stated that the proposed transcoding views (e.g., `utf8_view`) don't appear to function like other view adapters.
- Hubert agreed and noted that, unlike other view or iterator adapters, it appears that these views place additional semantic requirements on the iterator template parameters.
- Zach explained that, if `as_utf8()` is passed a pair of pointers to `char` that the iterator/sentinel pair is returned as is.
- Hubert noted that, for `utf8_view`, that makes `utf8_iter` an odd choice of name.
- Jens explained that requiring transcoding work to be performed by the provided `utf8_iter` is very surprising.
- Zach replied that the design is similar to `std::ranges::filter_view`; that view uses a filter iterator.
- Jens explained that there is a difference; `filter_view` defines a member type that functions as an iterator adapter and performs the work; in the proposed design, the user provided iterator does the work.
- Hubert noted that, if the provided iterator does all the work, then the view doesn't adapt anything.
- Corentin asserted that `utf8_view` does effectively the same thing that `std::ranges::subrange` does.
- Zach replied that there is a difference in that the transcoding views add a constraint that the provided iterator satisfy `utf_iter`.
- Fraser asked if it is intended that `as_utf8()` should be usable with, for example, a code page iterator that transcodes to UTF-8.
- Zach replied affirmatively.
- Corentin asserted that this design is a departure from prior ranges work.
- Hubert observed that the transcoding algorithms, unlike the transcoding iterators, do not allow for an error handler to be provided and asked Zach to confirm that they are intended to be less customizable and that programmers can use iterators in a loop to satisfy custom requirements.
- Zach confirmed and explained that the transcoding algorithms are intended to provide the functionality that is most often wanted, replacement character substitution, at high speed.

- Hubert stated that support for segmented data that is not necessarily aligned on a code unit sequence boundary requires a way to store a partial code unit sequence so that it can be completed by the following segment rather than substituting a replacement character for the partial sequence.
- Zach replied that a programmer could handle that requirement by checking that the segment ends with a complete code unit sequence by assigning an iterator to the end of the segment and decoding backwards.
- Jens suggested another approach would be to signal the end of the sequence early and then check if the base iterator has reached its end.
- Zach replied that adding additional semantics would complicate the usage experience.
- Hubert noted that, with regard to performance considerations, the work that JeanHeyd has been doing in WG14 offers more flexibility for error handling.
- Hubert asked if the transcoding iterator hierarchy unpacking is exposed such that a programmer could take advantage of it.
- Zach replied that he had not considered exposing it, but that doing so is a possibility that could be looked into.
- Zach agreed that it would be useful to have generalized support for such unpacking.
- Tom directed discussion towards the notion of using `char32_t` as a general Unicode code point type.
- Zach mentioned that his proposal uses `uint32_t`.
- Robin noted that ICU's `UCHAR` type is an alias of `int32_t`.
- Zach explained that the `value_type` of the UTF-32 iterators need only be a sufficiently sized integral type; not always a specific type like `char32_t`.
- Zach argued that the design should allow adaptation to the types the programmer is using.
- Jens asserted that the `value_type` of the UTF-32 converting iterators should be `char32_t` and that programmers can provide range wrappers for other types.
- Steve expressed support for always using `char32_t` because doing so enables overloading without ambiguity.
- Steve conveyed agreement with use of an adapter for other types.
- Fraser also expressed support for use of the `charN_t` types with the rationale that programmers should not be punished for using the right types.
- Fraser acknowledged that the design should not encourage casts.
- PBindels stated in chat, "I value new code getting warnings / errors on bad conversions, over legacy code getting support without casts."
- JeanHeyd reported that his implementation supports a configuration option that allows for use of a Unicode code point type other than `char32_t` and that every time he tries to exercise it that things break.
- JeanHeyd provided a link in chat to the documentation for the configuration option; <https://ztdtext.readthedocs.io/en/latest/config.html#config-ztd-text-unicode-code-point-distinct-type>.
- JeanHeyd added that, for older code bases, casts end up getting used regardless.

- JeanHeyd suggested that, if a choice has to be made, `char32_t` be preferred but with a possible opt-in option for use of another type by a minority of users.
- Zach stated that a template parameter could be added to types like `utf_8_to_32_iterator` to allow for a custom `value_type`.
- Zach asserted that the `charN_t` types are fine in some cases, but that the community currently uses `char` and `wchar_t` for UTF-8 and UTF-16 respectively and that it would be a shame to force the `charN_t` types on such users.
- Corentin stated that the choice of types accepted for input vs output are orthogonal.
- Corentin asserted that it makes sense to specify a single type for the Unicode code point type for many uses, such as for working with EGCs and argued that `char32_t` is the right type.
- Corentin noted that support for historical uses could hurt composability and stated that imposing questions of what type to use on programmers should be avoided.
- Corentin admitted that few people use `char8_t` and `char16_t` and stated that we can accept use of `char` and `wchar_t` with the caveat that programmers don't understand what the associated encodings are.
- Corentin expressed appreciation for the `charN_t` types having well-defined associated encodings.
- Jens argued that the library is too large as currently proposed and that it needs to be pruned and made more orthogonal.
- Jens insisted that we should be courageous; we are the standardization committee and we define what the future should look like.
- Jens noted that the standard containers were forward looking and the legacy container-like libraries are all gone now.
- Jens encouraged designing for composability by specifying elementary building blocks that can be combined.
- Jens asserted that it is ok if programmers are required to write a few wrappers for use with their existing projects.
- Zach asked what Jens would suggest cutting.
- Jens suggested removing the front, back, and insert iterators, perhaps in favor of a generic adapter that facilitates use with the existing front, back, and insert iterators.
- Jens added that the full matrix of UTF converters should be avoided in favor of conversion from one UTF encoding to a code point sequence and then to another UTF encoding.
- Jens expressed a desire for diagnostics on bad conversion as Peter Bindels mentioned in chat.
- Zach asked if that would imply issuing diagnostics for input provided in other integer types.
- Jens replied that a transform view can be used to explicitly support those cases.
- Jens stated that he wants a view that works more like other views; not like `as_utf8` in this proposal.
- Jens requested that range object adapters be proposed instead of functions.

- Jens explained that functions are problematic because of argument dependent lookup (ADL) and that range object adapters avoid that problem.
- Zach replied that his implementation uses customization point objects (CPOs), that the paper doesn't reflect that, but that it should.
- Victor stated that the project he works on contains a massive amount of code that assumes data held in `char`-based storage is UTF-8; that and similar projects require first class support for use of UTF-8 with `char`.
- Victor reported that his project has banned use of `char8_t` due to conflicts introduced when programmers tried to use it.
- Victor stated that, based on his basic understanding of Zach's proposal, the proposed design accommodates such usage.
- Tom asked Victor if such first class support could be conditionally provided based on the literal encoding being UTF-8.
- Victor replied that doing so could make sense.
- Victor indicated that, without good support for use of UTF-8 with `char`, his project would probably recommend against use of the facility.
- Corentin reported having considered implicit first class support for UTF-8 and `char` when the literal encoding is UTF-8, noted that such code is not portable, but expressed hope that such code would fail compilation rather than produce mojibake.
- Corentin suggested decreasing the size of the library by first focusing on a view and then adding eager converters and inserters later.
- Corentin expressed appreciation for the focus on UTF encodings vs support for all encodings.
- Tom stated that his and JeanHeyd's prior work on a `text_view` type that uses codecs satisfied a number of the composability concerns that Jens raised.
- JeanHeyd agreed that codecs provide more flexibility.
- JeanHeyd noted that bulk speed is attainable with his, Tom's and Zach's designs.
- JeanHeyd stated that codecs impose performance overhead relative to iterators due to state management but that good QoI can mitigate those costs.
- JeanHeyd observed that `short`, `int`, `wchar_t`, and other types have historically been used for text because the `charN_t` types weren't available.
- JeanHeyd asserted that whether and how programmers migrate to `charN_t` types depends on how difficult we make such a transition.
- JeanHeyd suggested that `charN_t` types be used by default due to their overloading abilities and strong encoding associations.
- JeanHeyd declared that, for `char` and `wchar_t`, one has to assume an encoding and, most of the time, that works out ok, but when it doesn't, it is a problem.
- JeanHeyd commented that the codec approach works well for handling `char` and `wchar_t`, but less well with iterators.
- JeanHeyd insisted that the missing library support for `charN_t` is an impediment.
- JeanHeyd reported that the work he is doing is more encompassing.
- Fraser asked whether anyone has written a paper regarding the pain points with using the `charN_t` types.

- Corentin replied that standard library support is missing.
- Jens provided specific examples of missing support; `printf()`, `std::format()`, and `std::iostreams`.
- JeanHeyd agreed that the most significant problem is the inability to conveniently print data held in those types.
- Corentin agreed that the lack of support for `charN_t` types in `std::format()` is a big issue.
- Steve replied that we would have to figure out what it means to print `char32_t`.
- Tom suggested we might be able to specify it for `std::print()`.
- Steve tentatively agreed but noted a need to consider locale impact.
- Victor agreed that locale issues need to be addressed.
- Steve noted that these are the reasons that Python 3 moved to the `c.UTF-8` locale by default.
- [*Editor's note: The Python 3 migration to the `c.UTF-8` locale was proposed in [PEP 538 \(Coercing the legacy C locale to a UTF-8 based locale\)](#).]*
- JeanHeyd noted that there is still a lot of code in the wild that assumes ASCII.
- Tom announced that the next meeting will be on April 12th and that we'll continue discussion of this paper.
- Tom suggested we should review the prior work on `text_view` given how long it has been since we've discussed it.

April 12th, 2023

Draft agenda:

- [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#):
 - Continue discussion.

Attendees:

- Charlie Barto
- Corentin Jabot
- Fraser Gordon
- Jens Maurer
- Nathan Owens
- Peter Brett
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2728R0 \(Unicode in the Library, Part 1: UTF Transcoding\)](#):
 - Zach spoke to concerns raised in the previous discussion regarding the need for output iterators:
 - There is a need for programs that use UTF-8 internally to convert to and from UTF-16 in `wchar_t` for Windows and `char16_t` for ICU.
 - The proposed "out" and "insert" transcoding iterators implement a push model; the others implement a pull model.
 - The "out" and "insert" transcoding iterators are useful to store the output of an eagerly evaluated pipeline.
 - PBrett stated that, for an eager algorithm, the size of the range is known.
 - Zach disagreed that the final size is necessarily known, but noted that it is often possible to predict an approximate size.
 - Tom asserted that output iterators don't work for transcoding since a failure to assign a complete code unit sequence results in silent data loss or UB or similar.
 - Zach suggested that the iterator destructors could perform some kind of flush operation.
 - Zach agreed that the output iterators could be removed from the proposal but that he had encountered situations where he couldn't both use a view and efficiently compute an output size.
 - Jens asserted that the existing `std::back_insert_iterator`, `std::front_insert_iterator`, and `std::insert_iterator` types should suffice for the proposed back insert, front insert, and insert iterators.
 - Jens observed that there has so far not been much demonstrated motivation for push-based iterators but stated that a general output iterator abstraction should be developed if compelling use cases are identified.
 - Jens recalled prior consensus for specifying Unicode algorithms that operate on code points rather than on code units.
 - Jens concluded that the output of any kind of eager algorithm should therefore be a sequence of code points that are then piped into encoding iterators.
 - Jens stated that he is strongly opposed to adding the full matrix of transcoding iterators.
 - Corentin requested stronger motivation for transcoding output iterators.
 - Corentin noted that the size of a range that has iterators that only model `std::input_iterator` is never known prior to iterating it.
 - Corentin observed that non-sized ranges exist in C++23 and programmers have so far been ok with that.
 - Corentin suggested that `std::ranges::to()` should suffice and should perform similarly to direct use of an output iterator.
 - Zach replied that output iterators are essentially never needed with ranges.
 - Corentin asked when an output iterator would be preferred over a range.

- Zach replied that performance experiments demonstrated that output buffers performed better for eager algorithms.
- Corentin expressed concern that output iterators add complexity but don't provide an order of magnitude performance improvement.
- Zach stated that output iterators are somewhat odd, but that they aren't particularly complicated.
- Corentin countered that their specification would still require spending additional time in wording review that would come at the expense of something else.
- Zach reiterated his willingness to drop them for now and to revisit later if needed.
- PBrett expressed support for deferring features that are not essential so as to narrow the proposal to the feature set that will provide the most value to the community.
- Zach stated that it is an option to not include eager algorithms at all but that doing so leaves performance on the table.
- Tom acknowledged that views are not well optimized today and suggested that might change in the future.
- Jens indicated low expectations with regard to teaching optimizers the peculiarities of views and noted the lack of improvements for, for example, string concatenation.
- Jens opined that implementors tend to be better off focussing on SPEC benchmarks.
- Jens explained that the incremental processing of view pipelines requires intermediate state that is difficult to lower to a vectorizable loop.
- Jens observed that vectorizing optimizers still lag the performance achieved by hand vectorized UTF-8 decoders.
- Zach agreed with Jens in chat; "I am just as skeptical as Jens about the optimization prospects."
- Jens stated that the paper would benefit from more rationale that motivates including the individual features in the C++ standard.
- Jens observed that we appear to have consensus to at least provide views.
- Jens expressed uncertainty whether it is reasonable to provide normalization as a view or whether an eager algorithm is required.
- Corentin agreed that lack of support for eager algorithms does leave performance on the table and estimated the difference as between 2x to 5x.
- Zach agreed, but argued the cost is closer to 2x.
- Corentin noted that the performance difference matters more in some cases; several MB of text might be needed before the difference becomes noticeable.
- Corentin asserted that quick and simple interfaces are needed in the standard to fill the existing functionality gap but that they don't need to be the fastest possible implementation.
- PBrett reported that [P2300 \(std::execution\)](#) includes a simple set of algorithms with an expectation that implementations will pattern match and optimize accordingly.
- PBrett asked whether implementors could provide, as a QoI concern, specialized implementations.

- Zach responded that such improvements are possible and reported that ICU avoids decoding for some operations.
- Zach stated that recognition of operations in pipelines will probably not be feasible.
- Tom suggested a case that can be specialized; a transcode algorithm where the inputs and outputs operate on contiguous storage.
- Zach agreed, but noted that dropping a chunk-by operation into a pipeline will prevent use of those kinds of specializations.
- PBrett observed that `std::ranges::to()` is always eager and, at the end of a pipeline, it seems technically possible to optimize based on the input and output types.
- Zach agreed and noted that library authors can handle that for simple cases.
- Jens noted that the ranges library is often proscriptive about types used and that use of features like `decltype` can interfere with specialization.
- Jens provided an example; the result of `std::ranges::views::take(R, n)` is a specialization of `std::span` if `R` is a `std::span`.
- Corentin reported having used ranges to optimize based on type in his prototypes.
- Corentin mentioned that text transformations tend not to produce sized ranges, but that their outputs usually have a size that is proportional to their input.
- Robin explained that, with regard to normalization, there is an upper bound of 3x on the possible size of the output, but acknowledged that is still well above the size usually required in practice.
- **Poll 1: SG16 would like to see a version of P2728 without eager algorithms.**
 - **Attendees: 10 (3 abstentions)**
 - **SF F N A SA**
4 2 0 1 0
 - **Consensus in favor.**
- Zach explained that, with regard to the `as_utfN()` factory functions behaving differently from those for other views, it is because they are defined in terms of any UTF iterator and a sentinel; this is what allows any iterator that produces UTF code units to be used.
- Jens expressed an expectation that an interface that requires a range of UTF-32 code points would use a concept that requires a range with corresponding constraints on its `value_type` and that programmers could provide their own views in that case.
- Corentin agreed.
- Tom stated that `as_utf8()` doesn't seem to actually do anything.
- Zach replied that it selects a transcoding iterator to implement transcoding from the UTF input iterator to UTF-8 and noted that the input iterator might not produce UTF-8 code units.
- Tom suggested a better name for such a function might be `to_utf8()`.
- Zach explained part of the motivation for use of the `utf_iter` concept as in `as_utf8()`; since the algorithms work on code points, its use enables implicit conversions that would otherwise require an explicit call to `as_utf32()` or similar.

- Jens acknowledged the utility of such conversions based on the `char8_t`, `char16_t`, and `char32_t`, but not for other types where the encoding is not clear.
- Jens requested that a revision of the paper include a view that behaves similarly to existing views in the standard.
- Jens also requested confirmation that normalization can be reasonably implemented as a view.
- Zach replied that he did not implement normalization or collation as a view.
- PBrett stated that collation can be performed by comparing two ranges.
- Zach replied that it doesn't make sense to implement reduction as a view.
- Zach stated that views didn't seem like a good match for normalization due to the state requirements.
- Corentin reported success having implemented normalization as a view.
- **Poll 2: UTF transcoding interfaces provided by the C++ standard library should operate on `charN_t` types, with support for other types provided by adapters, possibly with a special case for `char` and `wchar_t` when their associated literal encodings are UTF.**
 - Attendees: 9 (2 abstentions)
 - SF F N A SA
5 1 0 0 1
 - Consensus in favor.
 - SA: There is a precondition that input is intended to be UTF-8 and that isn't avoided by adding a wrapper; this doesn't help programmers to find bugs.
- **Poll 3: `char32_t` should be used as the Unicode code point type within the C++ standard library implementations of Unicode algorithms.**
 - Attendees: 9 (2 abstentions)
 - SF F N A SA
6 0 1 0 0
 - Strong consensus in favor.
- Tom announced that the next meeting will take place 2023-04-26 and will include review of:
 - [P2741R1: user-generated static_assert messages.](#)
 - [P2758R0: Emitting messages at compile time.](#)

April 26th, 2023

Draft agenda:

- [L2/23-107: Proper Complex Script Support in Text Terminals:](#)
 - Determine interest for participation in a potential new UTC project.
- [P2779R0: Make `basic\_string\_view`'s range construction conditionally explicit:](#)

- Determine whether to commit SG16 time to discussing this paper.
- [P2741R1: user-generated static_assert messages](#).

Attendees:

- Corentin Jabot
- Fraser Gordon
- Jens Maurer
- Mark de Wever
- Nathan Owen
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [L2/23-107: Proper Complex Script Support in Text Terminals](#):
 - Tom provided an introduction:
 - The paper authors seek to specify a protocol for the display of complex scripts in traditional text-based terminals.
 - Such a protocol could improve text formatting behavior beyond what has been achieved with [P1868 \(width: clarifying units of width and precision in std::format\)](#) and [P2675 \(LWG3780: The Paper \(format's width estimation is too approximate and not forward compatible\)\)](#).
 - The paper proposes the creation of a new project within the UTC.
 - Robin Leroy indicated that, should a new UTC project be approved, volunteers to participate in it would be welcome.
 - Anyone interested should reach out to Robin.
 - Fraser stated that the paper is an interesting read regardless of any interest in participation in a UTC project.
- [P2779R0: Make basic_string_view's range construction conditionally explicit](#):
 - Tom stated that, in his opinion, the paper does not raise SG16 concerns; however, the paper lists SG16 as an audience and Victor requested that SG16 review it.
 - Tom explained that the discussion today is only intended to determine whether SG16 should spend time on this paper prior to receiving a request from the LEWG chair.
 - Victor opined that SG16 should review because the paper proposes the use of `std::char_traits` to detect a string-like type.
 - Tom observed that the proposed wording doesn't actually reference `std::char_traits`.
 - Fraser asked for confirmation that the question in front of SG16 is whether to weight in on this use of `std::char_traits`.
 - Victor confirmed.

- Tom commented that the proposed wording only appears to use a traits type to name specializations of `std::basic_string_view` in order to opt them into the proposed `is_string_view_like` trait.
- Victor directed Tom to look at the wording for option 2.
- Corentin stated that he has been considering writing a paper to prohibit user specializations of `std::char_traits`.
- Zach commented that traits types can be awkward; especially with SFINAE.
- Tom observed that the proposed wording only checks whether both types have a matching member type named `traits_type`; it doesn't check for `std::char_traits` specifically.
- Corentin requested a poll to encourage LEWG not to rely on `std::char_traits`.
- Tom asked if anyone was opposed to such a poll.
- Jens noted that the proposal just performs a type tag comparison and doesn't inspect the type definition.
- Jens emphasized that this is just a heuristic intended to identify types that are semantically similar to `std::string_view`.
- Jens commented that if a better heuristic were to be found, that would be great, but otherwise the proposed heuristic seems conservatively correct.
- Jens stated that he does not see an SG16 concern here.
- Zach expressed a preference towards waiting for the paper author to present before any polls are taken.
- Zach stated that the `traits_type` name is not great for enabling this heuristic, but is not as bad as examining `std::char_traits` directly.
- Corentin suggested that option 2 might not be needed given the option 1 approach to explicitly opt in using a different name.
- Jens noted that the proposed wording for option 1 subsumes option 2.
- Victor noticed that the paper lists Folly's `fbstring` type and asserted that the implicit opt-in would not be wanted by its maintainers.
- Victor expressed a desire to deprecate `std::char_traits`.
- Tom agreed with Zach that the paper author should be granted an opportunity to present his perspective before SG16 takes further action.
- Tom said he would reach out to the paper author to see if he would like to present.
- [P2741R1: user-generated static_assert messages](#):
 - Corentin introduced the paper:
 - The paper proposes an extension to `static_assert` to enable the optional message to be provided by a constant expression evaluated at compile-time.
 - Barry Revzin's [P2758R0 \(Emitting messages at compile time\)](#) proposes some additional `constexpr` functions that do similarly.
 - The functionality these papers propose is not an SG16 concern, but the encoding used for the messages is.
 - The only encoding that is currently known at compile-time for strings in char-based storage is the ordinary literal encoding.

- There is a question of whether the proposed features should also support `wchar_t`, `char8_t`, `char16_t`, and/or `char32_t`.
- Victor noticed that the paper contains an example that uses `std::format` in an expression that requires constant evaluation despite the current lack of `constexpr` support for it in the standard and no current proposal to add such support.
- Corentin acknowledged that limitation but noted that the `format` implementation in [libfmt](#) has such support.
- Corentin explained that the proposed addition to `static_assert` is that the message parameter accept string-like types that have appropriate `data()` and `size()` member functions.
- Corentin noted that this suffices to enable any string produced during constant evaluation to be provided by wrapping it in a `std::string_view`-like type.
- Corentin stated that there is an open question of whether string-like types with non-contiguous storage should be supported.
- Corentin added that, as proposed, text in `char8_t`-based storage can also be used.
- Zach asked why `std::format` doesn't support constant evaluation.
- Corentin replied that the question should be directed to EWG, but noted some existing constant evaluation limitations; that floating-point types aren't supported for example.
- Zach asked what an implementation would do when trying to present the message to a user.
- Corentin responded that it would have to convert the message from the literal encoding to the encoding used to display text to a user.
- Corentin noted that this would add a new conversion requirement to compilers.
- *[Editor's note: Implementations are currently required to convert from the source file encoding to the various literal encodings, but do not necessarily need to be able to convert from those literal encodings to any other encoding.]*
- Zach commented that the proposed wording makes the design clear and that he has no concerns.
- Victor stated that `std::format` could support `constexpr` now given that it has been shown to be implementable.
- Victor asserted that the ordinary literal encoding is the right choice for text in `char`-based storage.
- Victor commented that the only question for SG16 is whether to add support just for `char` or for all of the character types.
- Jens observed a parsing issue in the proposed wording that will require disambiguation; a string literal matches both *unevaluated-string* and *constant-expression*.
- Jens suggested that, if text in `char8_t`-based storage is supported for the *constant-expression* case, then UTF-8 string literals should also be supported for the *unevaluated-string* case so that such literals don't fall into the former case.
- Corentin suggested that EWG should decide whether pointers to null-terminated strings should be supported.

- [Editor's note: Discussion ensued regarding unevaluated strings, UCNs, conversion to literal encodings, and whether two grammar productions are really required; the editor failed to record an accurate record of the discussion.]
- Jens requested that the wording be rebased on the current working draft.
- Jens noted that the pointer+size interface is preferred for the constant evaluation case and that therefore creates motivation for treating string literals as a special case.
- Jens noticed that the proposed wording suggests that the *constant-expression* argument is evaluated multiple times.
- Jens identified an additional wording issue; "possibly const-qualified type is `char*` or `char8_t*`" should be something like "pointer to possibly const-qualified `char` or `char8_t`."
- Mark asked Victor if the `libfmt` format implementation supports floating-point types in constant evaluation.
- Victor confirmed that it does, but that it doesn't use `to_chars()` to do so.
- Corentin spoke towards the motivation to support character types other than `char`:
 - Support for `u8""` and `char8_t` ensures the availability of an encoding that supports all characters.
 - Support for `L""` and `wchar_t` enables the use of existing `constexpr` string building functions used on Windows.
- Tom asked if it is necessary to allow encoding prefixes for the *unevaluated-string* case and noted that [P2361 \(Unevaluated strings\)](#) argued that such allowances are not needed.
- Corentin replied that the only motivation would be to prevent such expressions from falling into the *constant-expression* case.
- Jens noted that falling into that case would render the program ill-formed since string literals don't have a type that satisfies the requirements for `data()` and `size()` members.
- Tom responded that the *constant-expression* case could also support pointers to null-terminated strings.
- Zach asked if the requirements could be specified in terms of `std::data()` and `std::size()`.
- Corentin replied that such support would require including standard headers.
- Jens observed that use of `std::data()` and `std::size()` with the arrays produced by string literals would create ambiguity regarding the presence or absence of a null terminator in the string data.
- Zach lamented the absence of a trait to differentiate string literals from other kinds of expressions.
- Jens replied that any such solution would require extensions into the type system.
- Jens reported that the last line of the proposed wording refers to a *string-literal* that is not present in the *static_assert-declaration*.
- Victor expressed a preference for avoiding null-terminated strings and sticking to the proposed pointer+size design.

- Jens requested that the wording be aligned across other declarations with regard to the requirements to display messages at compile-time and advised discussing with the author of [P1301](#) ([\[\[nodiscard\("should have a reason"\)\]\]](#)).
- **Poll 1: Text produced during constant evaluation in char-based storage that is provided to the compiler shall be encoded in the literal encoding.**
 - Attendees: 6
 - SF F N A SA
2 4 0 0 0
 - Strong consensus in favor.
- **Poll 2: static_assert(cond, constant-expression) should support expressions that produce a range of char result.**
 - Attendees: 6
 - No objection to unanimous consent.
- **Poll 3: static_assert(cond, constant-expression) should support expressions that produce a range of wchar_t result.**
 - Attendees: 6
 - SF F N A SA
0 1 1 4 0
 - A: Would like to see more motivation.
 - Consensus against.
- **Poll 4: static_assert(cond, constant-expression) should support expressions that produce a range of char8_t result.**
 - Attendees: 6
 - SF F N A SA
1 1 1 2 1
 - SA: We shouldn't complicate the feature; char should suffice.
 - No consensus.
- **Poll 5: static_assert(cond, constant-expression) should support expressions that produce a range of char16_t or char32_t result.**
 - Attendees: 6
 - SF F N A SA
0 0 3 3 0
 - No consensus.
- Tom stated that the next meeting will be on 2023-05-10 and that an agenda is yet to be determined.

May 10th, 2023

Draft agenda:

- [P2728R3 \(Unicode in the Library, Part 1: UTF Transcoding\)](#).

Attendees:

- Corentin Jabot
- Eddie Nolan
- Fraser Gordon
- Nathan Owens
- Jens Maurer
- Peter Brett
- Robert Leahy
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- A round of introductions was held in honor of a new attendee, Eddie Nolan.
- [P2728R3 \(Unicode in the Library, Part 1: UTF Transcoding\)](#):
 - PBrett introduced the topics for today.
 - Zach provided an introduction to the latest revision:
 - Use case 4 in section 4.4 illustrates use with `std::print()` and `std::cerr`.
 - The `code_unit` concept still relies on `sizeof()` pending changes to infer encoding based on `charN_t` types; that change is still in progress awaiting fixes to existing tests.
 - Victor pointed out an error in section 4.4; the call to `std::print()` is missing the required format string.
 - Corentin asked how encoding is handled for the `operator<< ostream inserter`.
 - Zach responded that `as_utf8` produces a `utf_view` specialization for which specializations of `std::formatter` and overloads of `operator<<` are defined.
 - Zach noted that the current paper revision specifies `utf_view` but that prior revisions specified distinct templates for `utf8_view`, `utf16_view`, and `utf32_view`.
 - Zach explained that `utf_view` is able to produce UTF-8 from whatever encoding it adapts.
 - Corentin stated that some kind of string-like type is needed to enable formatting.
 - Zach noted the potential for additional views such as a `toupper_view` that performs case conversions.
 - Corentin responded that it should not be necessary to provide `operator<<` overloads for every view; a generic formatting mechanism is needed.
 - Zach explained that the goal is to provide interoperation with streaming and formatting and asserted that we need a convenient way to format these types.
 - Zach agreed that a `printable_view` or similar could be provided, but asserted it would be preferable to just be able to format them directly.

- Victor stated that generic formatters can be provided assuming a mechanism to determine the proper source and destination encoding.
- Victor observed that, when the changes to infer encoding based on `charN_t` types is complete, that there won't be any support for `char` and `wchar_t`.
- Zach confirmed the observation and noted this reflects prior consensus in prior polls.
- Robin posted the text of the relevant poll in the chat:

Poll 2: UTF transcoding interfaces provided by the C++ standard library should operate on `charN_t` types, with support for other types provided by adapters, possibly with a special case for `char` and `wchar_t` when their associated literal encodings are UTF.

- PBrett interpreted the poll as meaning that the author can choose to provide support for `char` and `wchar_t` as code units when the associated literal encoding is a UTF encoding.
- Zach suggested those types could be supported with another adapter.
- Tom stated that, with regard to another adapter, there is a question of whether such an adapter should be implicitly used or require explicit syntax.
- Zach expressed disfavor towards dependence on the literal encoding for portability reasons.
- Tom acknowledged that existing use of the literal encoding to infer an encoding for `char`-based text is an imperfect heuristic.
- PBrett expressed support for having a `std::formatter` specialization that supports these views, but expressed discomfort with ostream support for `operator<<` since there is no indication of what encoding to use.
- PBrett asked what the proposed `operator<<` actually does.
- Zach replied that it prints octets one at a time.
- PBrett explained that `std::print()` doesn't necessarily just write octets.
- Zach asked if `std::print()` is equivalent to streaming the result of a call to `std::format()`.
- PBrett replied negatively.
- *[Editor's note: See the Effects description for `std::vprint_unicode()` in [\[print.fun\]p7](#): the `std::print()` family of functions have specialized behavior when the target stream is a Unicode capable terminal.]*
- Zach opined that stream support is very useful and suggested `operator<<` could be conditionally supported for non-ASCII based systems.
- Zach stated that he is most concerned with ensuring that `std::format()` and `std::print()` work as intended.
- Jens summarized the status quo; `std::print()` does not currently support `char8_t`-based text as either the format string or as a format argument.
- Jens observed that handling of `char8_t` encoded data as proposed is novel since it effectively passes `char8_t`-encoded data as `char`-based input to `std::print()`.
- Jens asserted that transcoding should be performed or the print attempt should be ill-formed.

- Jens acknowledged the encoding questions will certainly be revisited at some point.
- Corentin stated that the lack of support for `charN_t` in `std::format()` is a separate issue that awaits someone doing the work needed.
- Zach explained that the new paper revision moves `null_sentinel_t` from the `std::uc` namespace to `std`.
- Zach noted that `null_sentinel_t` equality is determined by comparison with a value constructed object of the other type.
- Zach asserted that `null_sentinel_t` is important for support of string literals.
- Corentin suggested that `null_sentinel_t` could be split off to a separate paper for SG9 and EWG to review since it doesn't need to be tied to Unicode.
- PBrett observed that there is an implicit vs explicit tradeoff with regard to `null_sentinel_t`; some functions take sized ranges and support embedded null characters while others don't.
- PBrett noted this presents the possibility of a string being inadvertently truncated.
- Zach agreed that unintended truncation can occur with `utf_view`.
- PBrett asked to confirm that such truncation can only occur when a single pointer is passed to the constructor.
- Zach confirmed and directed attention to the `utf_range_like` concept.
- Victor stated that `null_sentinel_t` makes sense as a generic facility but that the proposed `base()` member function seems specific to this use case.
- Zach agreed that the member is specific to the ability to navigate a hierarchy of transformed ranges.
- Steve agreed that something like `null_sentinel_t` is needed to have a good story for support of null terminated strings.
- Steve stated that requiring explicit syntax just adds noise if practical use requires having to be explicit all the time.
- Tom asked if ranges already has a concept for layered range transformations and navigation between them; if so, SG9 could comment on the usefulness of the proposed `base()` member function.
- Zach noted that views often provide a `base()` member function; likewise with iterators such as `reverse_iterator`.
- Tom asked if there is precedent for a range-or-iterator concept like `utf_range_like`.
- Jens replied negatively.
- Tom suggested taking the `base()` and range-or-iterator concept concerns to SG9 to see if they have comments or if there is existing practice that we should align with.
- Jens stated that `null_sentinel_t` seems useful but that it should be moved to the `std::ranges` namespace.
- Jens argued that there is no need to move `null_sentinel_t` to a separate paper now; that can be done later if it would be helpful.
- Jens requested the removal of the `base()` member since there is no base to begin with and noted that there are other means to accomplish the purpose it is intended to serve.
- Jens stated that any chaining will be on the range level rather than the iterator level.

- Corentin agreed with Jens.
- Corentin noted that sized range types like `std::string_view` are advantageous since the size is maintained; calls to `strlen()` are avoided.
- Zach stated that Eric Niebler has demonstrated the effectiveness of sentinels.
- Zach commented that Jens is probably right regarding removal of the `base()` member.
- **Poll 1: Separate `std::null_sentinel_t` from P2728 into a separate paper for SG9 and LEWG; SG16 does not need to see it again.**
 - **Attendees: 12 (3 abstentions)**
 - **SF F N A SA**
1 1 4 2 1
 - **No consensus; author's discretion for how to continue.**
- Zach began reviewing the proposed constants and utility functions in section 5.4.
- Zach stated that `replacement_character` is useful to have.
- Zach explained that the `starts_encoded()` and `ends_encoded()` functions are just for convenience.
- Tom asked why `starts_encoded()` requires a range rather than supporting the range-or-iterator approach offered with `utf_range_like` and what the criteria is for determining when the range-or-iterator approach should be used.
- Jens expressed a preference that these remain range-only.
- Zach agreed with Jens and explained that these are low level functions that don't require a high level of ergonomics.
- PBrett noticed that there is an `is_low_surrogate()` function, but that a corresponding `is_high_surrogate()` is absent and argued that neither or both should be provided.
- Steve stated in chat: "If we don't provide `is_high_surrogate` we have to explain that forever, even if it's low value."
- Zach reported a mistake in use case 3 in section 4.3; `is_lead_code_unit()` is used, but is no longer proposed with the other utility functions.
- Corentin stated that the proposed utility functions provide a subset of Unicode character properties that programmers don't really want to know about.
- Corentin suggested that, if support for use case 3 is desirable, it might be better to provide a type that maintains the required state internally instead of users having to use `is_lead_code_unit()` and such on their own.
- Fraser suggested changing the names to `is_lead_surrogate()` and `is_trail_surrogate()` because it is difficult to remember what is meant by high and low.
- Victor stated in chat: "I'd prefer to stick with established terminology and not invent new terms, even if they are marginally clearer."
- Robert agreed with Victor in chat.
- Jens offered a quote from Unicode 15 section 3.8:

D71 High-surrogate code point: A Unicode code point in the range U+D800 to U+DBFF.
...

D73 *Low-surrogate code point*: A Unicode code point in the range U+DC00 to U+DFFF.

- Fraser countered with a quote from from later in the same section:

D75 *Surrogate pair*: ...

- ...
- Sometimes high-surrogate code units are referred to as *leading surrogates*. Low-surrogate code units are then referred to as *trailing surrogates*. This is analogous to usage in UTF-8, which has *leading bytes* and *trailing bytes*.
- ...

- Jens opined that a clear scope for the utility functions is needed in order to avoid design-by-committee.
- Jens suggested that `is_scalar_value()` and `is_code_point()` might be appropriate, but noted that `is_assigned_code_point()` would not be an elementary level building block.
- Jens requested an analysis of what building blocks should be exposed.
- Jens stated that the incremental read of a UTF-8 stream use case is interesting and can be in scope, but that more design and analysis is needed.
- Jens asserted that the paper needs more rationale of what the scope is and what the criteria is for inclusions and exclusions.
- Steve indicated that the proposed functions are facilities that he needs, but agreed with Jens that the streaming use cases probably deserve their own paper.
- Steve offered to work with Zach to move these functions into a separate classification paper if Zach is interested.
- Zach agreed that doing so sounds like the right approach.
- PBrett stated that `find_invalid_encoding()` would be more useful if it returned the range of invalid code units rather than just an iterator to the first.
- Zach agreed and noted that would be useful for replacement character handling.
- Tom stated that there are multiple replacement strategies to be considered.
- [*Editor's note see [PR-121](#) for a description of replacement character policies that conform to the Unicode standard.*]
- Zach stated that he would probably remove all of the utilities except for `replacement_character`.
- Tom responded that error policies need more discussion and suggested that `replacement_character` can likely be postponed as well.
- Corentin noted that a replacement character policy is required since Unicode algorithms only operate on code points.
- Zach explained that the transcoding iterators now adapt the iteratory category correctly.
- Zach noted that the complete set of UTF transcoding iterators is still present.
- PBrett asked what the `const char*` parameter of `use_replacement_character::operator()` is for.

- Zach explained that an error message is passed to the function.
- PBrett requested that the range of invalid code units be passed so that the entire sequence can be preserved in a thrown exception.
- Jens acknowledged the need for the transforming iterators to implement the transcoding view, but stated a lack of motivation for exposing them.
- Tom agreed and requested that, if there are strong use cases for the iterators, to please include the motivation for them in the paper.
- Zach acknowledged that the iterators can be obtained from the view type.
- Jens opined that the iterators seem like a step backwards given the effort to move towards range-based designs.
- Corentin stated that the need to store three iterators within each transcoding iterator to track the beginning and end of the range as well as the current position is novel; iterators generally don't know where the beginning and end of a range are; ranges maintain that information.
- *[Editor's note: Further discussion regarding iterators that hold references to a view occurred on the SG16 mailing list. See the [2023-05-10 dated messages in the thread with subject "Re: \[SG16\] New revision of P2728".](#)]*
- Tom reported a potential ambiguity with the range-or-iterator approach; string literals are arrays that satisfy range concepts but that also decay to a pointer that satisfies iterator concepts.
- Tom indicated that an explicit deduction guide might be needed to ensure that `utf_view` works as intended with CTAD.
- Tom and Zach agreed to take further discussion offline.
- PBrett stated that the next meeting will be on 2023-05-24 and that the agenda will include the following:
 - [P2779R0: Make basic_string_view's range construction conditionally explicit.](#)
 - Review of proposed deprecations by Alisdair (awaiting paper).